Arb Finder

Cross-market arbitrage scanning for Taiwan warrants & options — build narrative and EDA methodology

A pitch & exploratory-data-analysis report reconstructed from the commit history of a local trading-desk app. Scope: the **Arb Finder** engine only (warrant and option *scanners* excluded).

1 What the Arb Finder does

The app is a local Flask desk tool that pulls live Taiwan warrant quotes (CMoney), Taiwan equity-option quotes (TAIFEX/MIS), and US ADR option chains, then hunts for mispricings between instruments that reference the **same underlying economic exposure**. It grew into four scanning modes, each a different way two legs can be the 'same' asset:

- **Direct Match** — TW warrant vs TW equity option on the *same* local stock (same market, same currency).
- **US Option Match** — TW warrant vs a US-listed *ADR* option on the same company (two markets, two currencies, one company).
- **TW Option / US Option** — a TAIFEX option vs a US ADR option: a pure cross-market *basis* trade.
- **Straddle Vol Arb** — relative-value on *implied volatility*: long the cheapest-vol call+put package, short the dearest.

Every mode shares a spine of Taiwan-warrant mechanics that the pricing math must respect — get these wrong and every number downstream is off:

```
warrants trade in board lots (zhang) of 1,000 units; a quote P is per unit (one lot costs P×1,000); the exercise ratio is shares delivered per unit (often fractional). TW equity options are European, priced per share, contract size 2,000. US ADR options are American (early-assignment risk on a short leg).
```

Core normalization used everywhere

```
price per underlying share = warrant_unit_price / exercise_ratio
warrants (units) to cover N shares = N / exercise_ratio
board lots (zhang) = (N / exercise_ratio) / 1000
one option contract = 2000 shares → warrants_needed = 2000 / ratio
```

Normalizing both legs to a per-underlying-share price is what makes a warrant and an option directly comparable in the first place.

Build arc (from the commits)

The engine started narrow and got progressively more honest about execution risk. The commit trail shows the real process — not a clean spec, but a sequence of 'ship a matcher, discover a way it lies, add a guard':

- `b1fb21c` first cut: match TSMC warrants to the closest option by strike/expiry.
- `307791a` → `6313e75` add Put-Call-Parity mode + a trade-breakdown modal with a P&L chart.
- `4409023` **trust only live bid/ask** — drop settlement-price fallbacks that faked fills.
- `d6c39af` restrict to bull spreads (opt_strike ≥ warrant_strike) so the residual can't go negative.
- `29a62f3` **bug**: option contract size hard-coded 1000 instead of 2000 — every hedge size was 2× off.
- `33acce3` express the hedge in whole board lots + toggle exact-vs-whole-lot P&L (residual delta from rounding).
- A cluster of *reverts* around the 'no-naked-short / long-leg-expires-last' rule (`3ff74f3, 1c1b23a, 22a3ef7, 06eda3f`) — the DTE-guard logic was genuinely hard and took several passes.

2 Direct Match — warrant vs option, same underlying

Two instruments on the same local stock, same currency, so the only thing to reconcile is strike, expiry and the per-share price. Two engines run here.

2.1 Price-gap matcher (vertical-spread residual)

For each warrant, scan same-type options and keep every pair whose executable prices leave a locked gap. The scan runs in two directions:

```
positive (buy warrant / sell option): price_diff = opt_bid_share − warrant_ask_share > 0
negative (buy option / sell warrant): price_diff = opt_bid_share − warrant_ask_share < 0

guards: positive ⇒ opt_strike ≥ warrant_strike AND opt_dte ≤ warrant_dte
        (short leg must never outlive the long hedge → no naked short)
```

- **EDA here is combinatorial, not statistical**: enumerate the full warrant×option grid, normalize both legs to per-share, and filter on the sign of the gap under real quotes.
- **Loose toggle** (`63e3bdc, 4bbf9ca`): price the two tradable legs at their *favorable* side (buy@bid, sell@ask) to see gaps that a spread-crossing fill would erase — a sensitivity view, flagged non-executable.
- The kept residual is a bounded **vertical spread**; the modal draws its true at-expiry payoff from intrinsic value of both legs (`c3256a0, e8b48cc` dropped the misleading BS curve).

2.2 Put-Call-Parity matcher (synthetic replication)

Instead of matching like-for-like, price the warrant against a **synthetic** built from the opposite-type option + underlying + a bond. European PCP (no dividends):

```
C − P = S − K·e^(−rT)

synthetic call = P + S − K·e^(−rT)
synthetic put = C − S + K·e^(−rT)

edge: price_diff = synthetic − warrant (warrant cheap vs replication ⇒ buy it, sell
synthetic)
```

- Needs an **implied volatility** and a discount rate. IV is solved per leg from the live quote by Brent's method on the Black-Scholes price:

```
d1 = [ ln(S/K) + (r + ½σ²)T ] / (σ√T)  ;  d2 = d1 − σ√T
call = ratio·[ S·N(d1) − K·e^(−rT)·N(d2) ]
put = ratio·[ K·e^(−rT)·N(−d2) − S·N(−d1) ]
σ_implied = brentq( BS(σ) − market_price = 0 )
```

- PCP got a **loose-price toggle** too (`da28048`), but only on the one executable side — warrants can't be shorted, so the 'short warrant / long synthetic' leg is emitted for debugging only.
- This same PCP engine is later re-pointed at a *US ADR* spot to become the cross-market matcher (§3.4).

3 US Option Match — TW warrant vs US ADR option

The richest section, and where the EDA gets genuinely statistical. A TW warrant and a US ADR option reference the **same company** but trade in different markets and currencies. They are *not* 1:1: the ADR carries a floating **premium/discount** to the local shares, and USD/TWD floats on top. That basis is the entire trade — so it has to be measured, not assumed.

Universe selection came first (`adr_universe_screen.ipynb`, `umc_adr_parity.ipynb`): only names whose ADR tracks the local share tightly enough are included; a persistent, large premium (e.g. TSM) is excluded. Started with UMC/2303, expanded to CHT/2412 with per-stock ADR ratios (`b3fdf50, 789f46b`).

3.1 The premium series (the raw signal)

```
premium_t = ( adr_usd_t / adr_ratio × fx_t ) / tw_close_t − 1
```

- Convert the ADR to TWD-per-local-share and compare to the local close; the % gap is the premium. FX is *baked into* the numerator, so the raw premium already embeds currency risk.
- Built as a 3-year daily series (cached 30 min) with the **current** point refreshed live intraday (`9ee4a18`) so the entry signal isn't stale.
- Visualized as a time series + a **daily-premium distribution histogram** (`31c6d53`) — the first look at how fat-tailed / mean-seeking the basis is.

3.2 Is the premium actually mean-reverting? (ADF + half-life)

The whole thesis is that the premium pulls back. That is a **testable** claim, so it gets an Augmented Dickey-Fuller unit-root test on the premium level (`9fff47d`):

```
ADF null H0: unit root (random walk, NOT mean-reverting)
p < 0.05 ⇒ reject H0 ⇒ premium is stationary / mean-reverting ✓

AR(1) speed: Δp_t = a + b·p_t■1 , ϕ = 1 + b (need b < 0)
half-life = − ln 2 / ln(1 + b) trading days
```

Half-life answers the practical question the ADF p-value doesn't: if the premium is stretched today, how many days to decay halfway back? — which must fit inside the trade's time-to-expiry to be usable.

3.3 Decomposing premium into FX vs residual basis

The premium mixes two risks: currency (USD/TWD) and company-specific US-vs-local sentiment. The EV must not double-count FX (`92c010a` fixed exactly that), so the premium move is split by an **exact log identity**:

```
r_p = daily premium log-move = r_f + r_b
r_f = FX log-move r_b = residual basis move = r_p − r_f
    ( r_b = Δln(adr_usd) − Δln(fx) )

unexplained_frac = Var(r_b) / Var(r_p) ← premium variance NOT from FX
R² = corr(r_p, r_f)² ← how tightly premium tracks FX
```

- The **option leg's moneyness is driven by r_b only** (the US-only basis); FX is priced as a separate, additive risk factor, not smeared into the option payoff.
- FX EDA panel (`2748c2b, 1b398af, 7aadc12`): latest rate + 30d change, a premium-vs-FX correlation scatter with best-fit line, and a cumulative **premium-isolated FX path** (the residual index) so you can see which risk is actually moving.

3.4 Conditional-scenario Expected Value

Early EV used the unconditional mean premium — wrong, because you enter *from today's regime*. The commits walk the fix: drop unconditional EV, condition on the starting regime, center on today's premium (`dd46fdc, 3406a5f, 4ab6080, 87407be`):

```
1. classify today's premium regime (spike / around-0 / drop)
2. from history, get k-day-ahead transition probs P(end regime | start)
3. each bucket → conditional-mean EXIT premium (FX-stripped, r_b applied)

E[PnL] = Σ_i prob_i · E[ PnL | regime_i ] + (separate) FX factor
```

- Each leg is valued at the horizon with the surviving leg exercised if ITM else sold at time value (`bc101fd`); loose mode prices the unwind loose too (`2c42c9d`).
- FX handled as its own scenario tree (stay / TWD-weakens / TWD-strengthens with conditional means), then combined with the premium EV rather than folded in twice.

3.5 Cross-market PCP + execution reality

- The Direct-Match PCP engine is re-pointed at the ADR spot (`1ee768d`): the synthetic is priced off the ADR in USD (r = US rate), and the premium *is* the edge.
- **Residual greeks panel** (`074b720`): net Delta + Vega across the non-1:1 legs, so leftover directional / vol exposure is explicit, not hidden.
- **Orderbook depth check** (`4b9456c, 2b64c4e, 15bb2eb`): test best-level fillability — a gap you can't fill at size isn't an arb.

4 TW Option / US Option — pure cross-market basis

Both legs are options (TAIFEX vs US ADR), so delta roughly cancels and what remains is the ADR-premium + FX basis — a cleaner version of §3 without the warrant's exercise-ratio bookkeeping (`c22c1a6`).

```
match: closest strike, then closest expiry (delta-match priority)
entry credit / share: sell_leg − buy_leg (must be > 0)
pcp_diff = +credit if Long TW / Short US , else −credit

guards: short leg expires ≤ long leg (no naked short)
       type-aware strike: short_call ≥ long_call , short_put ≤ long_put
       (rejects verticals with a negative min-payoff)
```

- Same premium/FX/EV engine as §3 drives the scenario P&L — the reused statistical spine.
- Guard commits (`1abba93, b8abaac, cb3e468`) are again the hard part: several passes to guarantee no naked-short window and no built-in-loss vertical.

5 Straddle Vol Arb — relative value on implied vol

The newest mode (`5f603fd`) and a different kind of arb: not price gaps but **volatility**. A 'package' is one call + one put on the same underlying (strikes within ΔK = a strangle, so packages carry net delta). Compare packages in **implied vol**, not price, to strip out the |F−K| and √T terms that dominate raw straddle prices.

```
package IV = ( IV_call + IV_put ) / 2
LONG the cheapest-vol package (warrant OR option legs, all bought)
SHORT the dearest-vol package (OPTION legs only — warrants can't be shorted)

edge = short_IV − long_IV (in volatility points)
```

- Every leg's IV is **recomputed from its own per-share quote with ratio = 1.0** so warrant and option vols sit on the same scale — the source feed's IV mis-scales by ~10× and is never trusted for a cross-instrument comparison.
- Per (strike, dte) only the best leg is kept: MIN buy-vol to go long, MAX sell-vol to write — collapsing many same-strike issuer warrants to their best quote.

Takeaways

- The build process is **adversarial with itself**: ship a matcher, find a way it reports a fake edge (settlement prices, wrong contract size, naked-short windows, uncrossable depth), add a guard. The reverts are the tell.
- EDA **deepens as legs decouple**: Direct Match is combinatorial; ADR modes need a measured, tested (ADF), decomposed (FX vs basis), regime-conditional model of the premium; Straddle mode moves the whole comparison into vol space.
- One statistical spine — the premium series, its stationarity test, its FX/basis split, its conditional-scenario EV — is **reused** across US Option Match and TW/US Option.

Reconstructed from 113 commits on `main`. Formulas transcribed from `warrant_logic.py`, `us_options_logic.py`, and `app.py`.